

Lab 3

Student: Preeti Pokharel Environment: Ubuntu Linux and WSL Date: September 21 2026

TABLE OF CONTENT

Task 1 The Long Running Process	3
Theoretical Concept	3
Flow Diagram	3
Source Code	3
Compilation and Verification	3
Summary	4
Task 2 Process Identity PID and PPID	4
Theoretical Concept	4
Flow Diagram	4
Source Code	4
Compilation and Verification	4
Summary	5
Task 3 Exit Codes and OS Feedback	6
Theoretical Concept	6
Flow Diagram	6
Source Code	6
Compilation and Verification	7
Summary	7
Task 4 Standard Input and Output	8
Theoretical Concept	8
Flow Diagram	8
Source Code	8
Compilation and Verification	8
Summary	8
Task 5 Conditional Execution and Termination	9
Theoretical Concept	9
Flow Diagram	9
Source Code	9
Compilation and Verification	10
Summary	10
Conclusion	11

List of Figures

Figure 1: Source code for the long-running process	4
Figure 2: Terminal verification for Task 1	4
Figure 3: Source code for displaying PID and PPID	5
Figure 4: Terminal verification for Task 2	5
Figure 5: Source code for exit status control	7
Figure 6: Terminal verification for Task 3	8
Figure 7: Source code for standard input and output	9
Figure 8: Terminal verification for Task 4	9
Figure 9: Source code for conditional execution and termination	10
Figure 10: Terminal verification for Task 5	11

Task 1 The Long Running Process

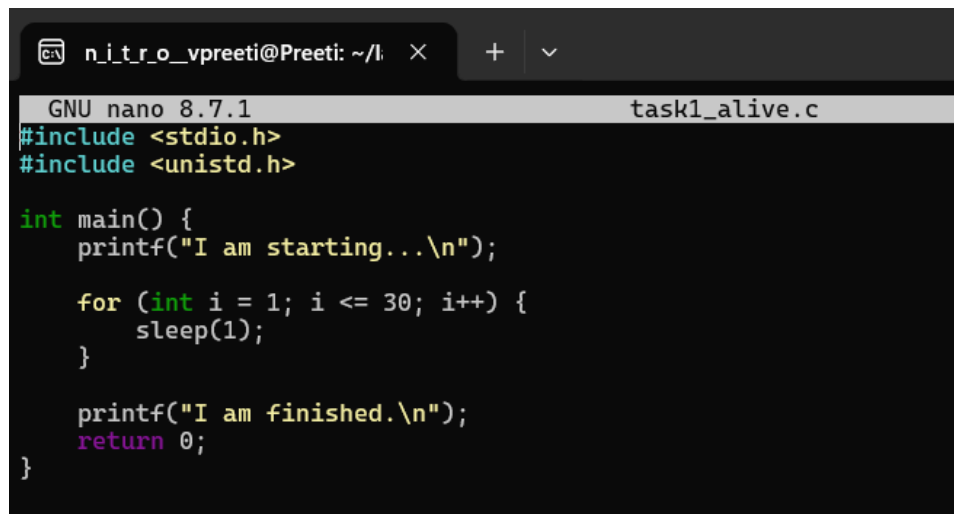
Theoretical Concept

A process is a program currently being executed. The sleep function pauses the process while it remains managed by the Linux kernel. Running the program with an ampersand places it in the background so the shell can accept another command.

Flow Diagram



Source Code



```

GNU nano 8.7.1 task1_alive.c
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("I am starting...\n");

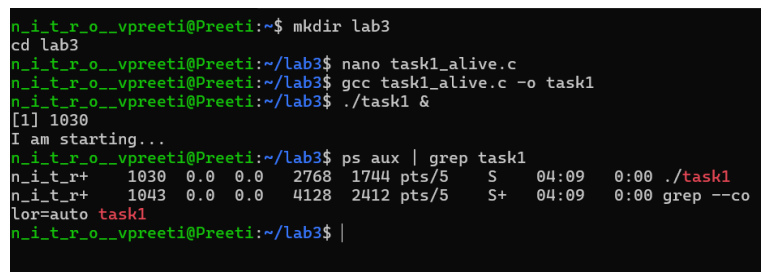
    for (int i = 1; i <= 30; i++) {
        sleep(1);
    }

    printf("I am finished.\n");
    return 0;
}

```

Figure 1: Source code for the long-running process

Compilation and Verification



```

n_i_t_r_o_vpreeti@Preeti:~$ mkdir lab3
n_i_t_r_o_vpreeti@Preeti:~$ cd lab3
n_i_t_r_o_vpreeti@Preeti:~/lab3$ nano task1_alive.c
n_i_t_r_o_vpreeti@Preeti:~/lab3$ gcc task1_alive.c -o task1
n_i_t_r_o_vpreeti@Preeti:~/lab3$ ./task1 &
[1] 1030
I am starting...
n_i_t_r_o_vpreeti@Preeti:~/lab3$ ps aux | grep task1
n_i_t_r+  1030  0.0  0.0  2768 1744 pts/5    S   04:09   0:00 ./task1
n_i_t_r+  1043  0.0  0.0  4128 2412 pts/5    S+  04:09   0:00 grep --co
lor=auto task1
n_i_t_r_o_vpreeti@Preeti:~/lab3$ |

```

Figure 2: Terminal verification for Task 1

Summary

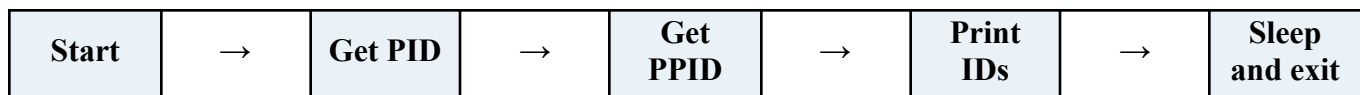
The program started, appeared in the process list while sleeping, and later printed I am finished.

Task 2 Process Identity PID and PPID

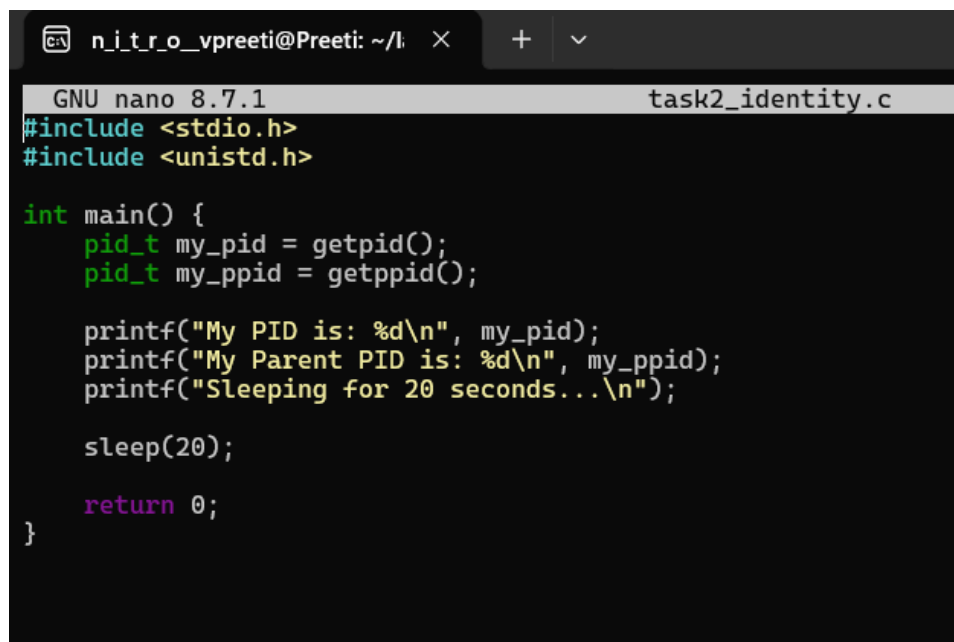
Theoretical Concept

Linux assigns every process a unique Process ID. The Parent Process ID identifies the process that launched it, normally the current shell. The getpid and getppid system calls query these identifiers.

Flow Diagram



Source Code



```

GNU nano 8.7.1 task2_identity.c
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t my_pid = getpid();
    pid_t my_ppid = getppid();

    printf("My PID is: %d\n", my_pid);
    printf("My Parent PID is: %d\n", my_ppid);
    printf("Sleeping for 20 seconds...\n");

    sleep(20);

    return 0;
}

```

Figure 3: Source code for displaying PID and PPID

Compilation and Verification

```

n_i_t_r_o__vpreeti@Preeti:~/lab3$ nano task1_alive.c
n_i_t_r_o__vpreeti@Preeti:~/lab3$ gcc task1_alive.c -o task1
n_i_t_r_o__vpreeti@Preeti:~/lab3$ ps aux | grep task1
n_i_t_r+  514  0.0  0.0  4128 2488 pts/0    S+   04:19   0:00 grep --color=auto task1
n_i_t_r_o__vpreeti@Preeti:~/lab3$ nano task2_identity.c
n_i_t_r_o__vpreeti@Preeti:~/lab3$ gcc task2_identity.c -o task2
./task2 &
[1] 554
My PID is: 554
My Parent PID is: 324
Sleeping for 20 seconds...
n_i_t_r_o__vpreeti@Preeti:~/lab3$ ps -p PID_NUMBER -o pid,ppid,cmd
error: process ID list syntax error

Usage:
ps [options]

Try 'ps --help <simple|list|output|threads|misc|all>'
or 'ps --help <s|l|o|t|m|a>'
for additional help text.

For more details see ps(1).
[1]+  Done                  ./task2
n_i_t_r_o__vpreeti@Preeti:~/lab3$ ./task1 &
[1] 584
I am starting...
n_i_t_r_o__vpreeti@Preeti:~/lab3$ I am finished.
./task2 & pid=$!; sleep 1; ps -p $! ./task2 & pid=$!; sleep 1; ps -p $pid -o pid,ppid,cmd
[2] 637
[1] Done                  ./task1
My PID is: 637
My Parent PID is: 324
Sleeping for 20 seconds...
  PID    PPID  CMD
  637    324  ./task2
n_i_t_r_o__vpreeti@Preeti:~/lab3$

```

Figure 4: Terminal verification for Task 2

Summary

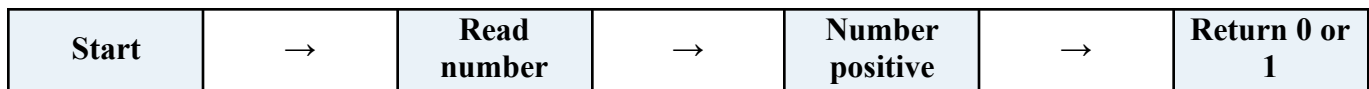
The program printed PID 637 and PPID 324. The ps output confirmed the same PID, PPID, and command.

Task 3 Exit Codes and OS Feedback

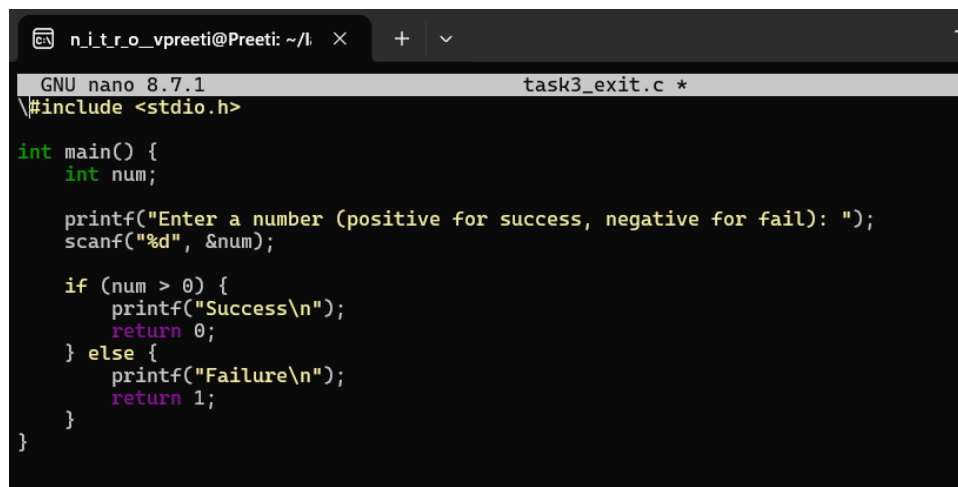
Theoretical Concept

A C program returns an integer status to the operating system. By convention, zero means successful completion and a nonzero value indicates failure. The shell variable \$? stores the status of the most recently completed command.

Flow Diagram



Source Code



```

n_i_t_r_o_vpreeti@Preeti: ~/l. × + v
GNU nano 8.7.1 task3_exit.c *
#include <stdio.h>

int main() {
    int num;

    printf("Enter a number (positive for success, negative for fail): ");
    scanf("%d", &num);

    if (num > 0) {
        printf("Success\n");
        return 0;
    } else {
        printf("Failure\n");
        return 1;
    }
}

```

Figure 5: Source code for exit status control

Compilation and Verification

```
PID      PPID  CMD
637      324  ./task2
n_i_t_r_o__vpreeti@Preeti:~/lab3$ nano task3_exit.c
[2]+  Done                  ./task2
n_i_t_r_o__vpreeti@Preeti:~/lab3$ gcc task3_exit.c -o task3
n_i_t_r_o__vpreeti@Preeti:~/lab3$ ./task3
Enter a number (positive for success, negative for fail): 6
Success
n_i_t_r_o__vpreeti@Preeti:~/lab3$ echo $?
0
n_i_t_r_o__vpreeti@Preeti:~/lab3$ ./task3
Enter a number (positive for success, negative for fail): 5
Success
n_i_t_r_o__vpreeti@Preeti:~/lab3$ echo $?
0
n_i_t_r_o__vpreeti@Preeti:~/lab3$ ./task3
Enter a number (positive for success, negative for fail): 5
Success
n_i_t_r_o__vpreeti@Preeti:~/lab3$ echo $?
0
n_i_t_r_o__vpreeti@Preeti:~/lab3$ ./task3
Enter a number (positive for success, negative for fail): -5
Failure
n_i_t_r_o__vpreeti@Preeti:~/lab3$ echo $?
1
n_i_t_r_o__vpreeti@Preeti:~/lab3$ |
```

Figure 6: Terminal verification for Task 3

Summary

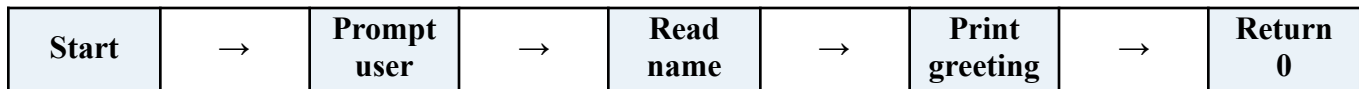
Positive inputs produced Success and exit status 0. Input -5 produced Failure and exit status 1.

Task 4 Standard Input and Output

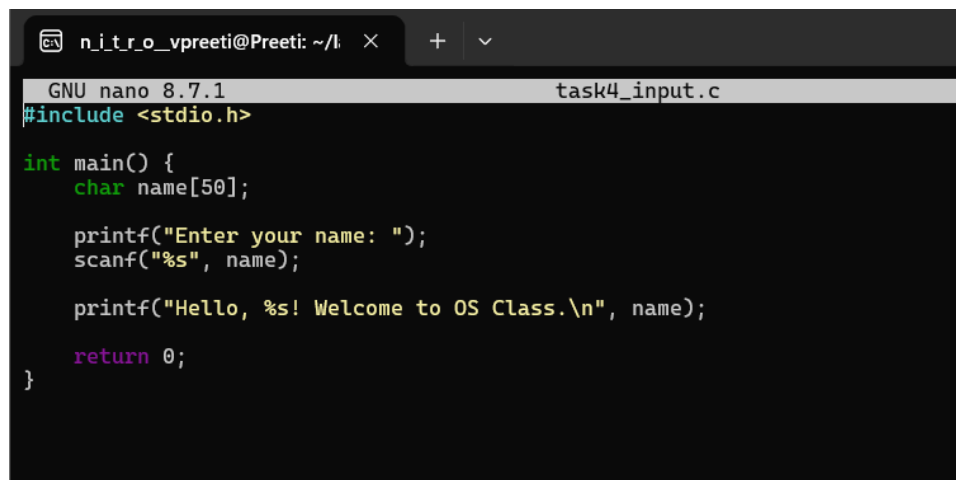
Theoretical Concept

The operating system provides standard streams. `scanf` reads from standard input, normally the keyboard, while `printf` writes to standard output, normally the terminal.

Flow Diagram



Source Code



```

GNU nano 8.7.1 task4_input.c
#include <stdio.h>

int main() {
    char name[50];

    printf("Enter your name: ");
    scanf("%s", name);

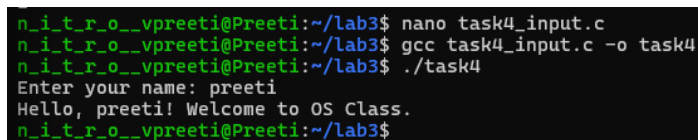
    printf("Hello, %s! Welcome to OS Class.\n", name);

    return 0;
}

```

Figure 7: Source code for standard input and output

Compilation and Verification



```

n_i_t_r_o__vpreeti@Preeti:~/lab3$ nano task4_input.c
n_i_t_r_o__vpreeti@Preeti:~/lab3$ gcc task4_input.c -o task4
n_i_t_r_o__vpreeti@Preeti:~/lab3$ ./task4
Enter your name: preeti
Hello, preeti! Welcome to OS Class.
n_i_t_r_o__vpreeti@Preeti:~/lab3$

```

Figure 8: Terminal verification for Task 4

Summary

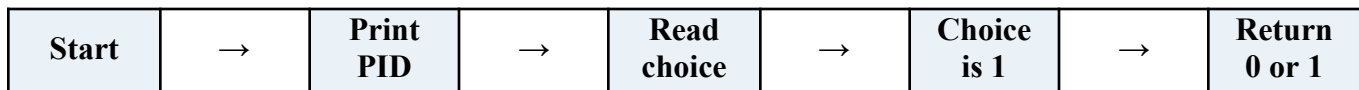
The program accepted the entered name and printed the expected welcome message.

Task 5 Conditional Execution and Termination

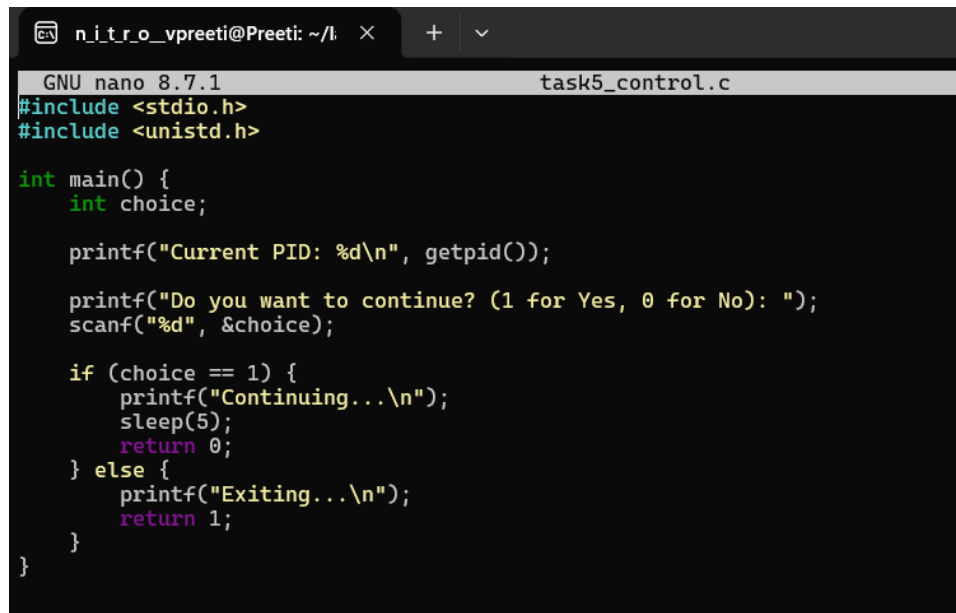
Theoretical Concept

Programs can select different execution paths based on input. This program prints its PID, reads a choice, and either continues for five seconds with status 0 or exits immediately with status 1.

Flow Diagram



Source Code



```

GNU nano 8.7.1 task5_control.c
#include <stdio.h>
#include <unistd.h>

int main() {
    int choice;

    printf("Current PID: %d\n", getpid());

    printf("Do you want to continue? (1 for Yes, 0 for No): ");
    scanf("%d", &choice);

    if (choice == 1) {
        printf("Continuing...\n");
        sleep(5);
        return 0;
    } else {
        printf("Exiting...\n");
        return 1;
    }
}

```

Figure 9: Source code for conditional execution and termination

Compilation and Verification

```
n_i_t_r_o__vpreeti@Preeti:~/lab3$ nano task5_control.c
n_i_t_r_o__vpreeti@Preeti:~/lab3$ gcc task5_control.c -o task5
n_i_t_r_o__vpreeti@Preeti:~/lab3$ ./task5
Current PID: 852
Do you want to continue? (1 for Yes, 0 for No): 1
Continuing...
n_i_t_r_o__vpreeti@Preeti:~/lab3$ echo $?
0
n_i_t_r_o__vpreeti@Preeti:~/lab3$ ./task5
Current PID: 873
Do you want to continue? (1 for Yes, 0 for No): 0
Exiting...
n_i_t_r_o__vpreeti@Preeti:~/lab3$ echo $?
1
n_i_t_r_o__vpreeti@Preeti:~/lab3$
```

Figure 10: Terminal verification for Task 5

Summary

Choice 1 continued and returned 0. Choice 0 exited and returned 1. Each run received a distinct PID.

Conclusion

The lab demonstrated that a C program becomes an operating system process when executed. Linux assigns the process an identity, schedules it during delays, provides standard input and output streams, and records the program termination status. The terminal evidence confirmed that PID and PPID values, background execution, branching, and exit codes behaved as expected.

Submission Checklist

Five correctly named C source files	Completed
Theoretical concepts	Included
Flow diagrams	Included
Source-code screenshots	Included
Terminal output screenshots	Included
README file	Prepared
Push code to personal GitHub repository	Completed